ROOK

Setup & Launch Guide

Everything you need to connect Supabase, Stripe, an AI provider, and job data sources to the ROOK codebase.

Before you start

This guide assumes you have the rook-project folder (the code) already — it walks through creating and connecting the outside services, not writing code. Where a step needs a value pasted into a file, it tells you exactly which file and which line.

What You'll Be Signing Up For

Three accounts, total. None of them require a business entity, an LLC, or a company bank account just to get started in test/development mode.

- Supabase — database, user accounts/login, and file storage for résumés. Free to start.
- Stripe — billing, so candidates can pay the $29/month subscription. Free to create; only takes a cut once you're charging real money.
- An AI provider (Anthropic or OpenAI) — powers résumé parsing, match explanations, and the embeddings used in matching. Pay-as-you-go, pennies at low volume.

About Greenhouse, Lever, and Ashby

You do NOT need to sign up for anything with these three. Their published-job endpoints are public and require no account or API key on your end — you just need to know each employer's board identifier. Section 5 below covers exactly how to find it.

1. Supabase Setup

1.1 Create the project

1. Go to supabase.com and sign up (GitHub login is fastest).
2. Click “New Project.” Choose an organization, name it rook-careers, set a strong database password (save it somewhere — you won't need it day to day, but you'll want it if you ever connect a raw Postgres client).
3. Pick the region closest to your users (US East is a reasonable default for a Florida-based audience).
4. Wait about two minutes for the project to finish provisioning.

1.2 Run the database schema

1. In the Supabase dashboard, open the SQL Editor (left sidebar).
2. Click “New Query.”
3. Open backend/db/schema.sql from the project folder, copy its entire contents, and paste it into the SQL editor.
4. Click “Run.” You should see “Success. No rows returned.” This creates every table ROOK needs: employers, jobs, candidate_profiles, candidate_job_matches, and applications.

1.3 Create the résumé storage bucket

1. In the sidebar, go to Storage.
2. Click “New bucket,” name it resumes, and leave it Private (not public) — résumés should never be publicly browsable.
3. That's it for the bucket itself; the schema's row-level security policies already restrict who can read what once real auth is wired in.

1.4 Get your API keys

1. Go to Project Settings (gear icon) > API.
2. You'll need three values: the Project URL, the anon public key, and the service_role secret key.

> **Keep the service_role key secret**
>
> The service_role key bypasses all row-level security. It belongs only in your backend's .env file, never in any frontend/browser code. The anon key is safe to use in the browser.

3. Copy all three into your .env file (see the Environment Variables section near the end of this guide).

2. Stripe Setup

2.1 Create your account

1. Go to stripe.com and sign up. You can fully build and test everything in Test Mode before ever providing real business/banking details.
2. Stay in Test Mode (toggle in the top-right of the dashboard) until you're ready to charge real customers.

2.2 Create the Professional plan product

1. In the dashboard, go to Product Catalog > Add Product.
2. Name it ROOK Professional.
3. Under pricing, choose Recurring, set the price to $29.00, and billing period to Monthly.
4. Save. Stripe will show you a Price ID that looks like price_1Nx... — copy this.
5. (Optional) Repeat for an annual price at $249/year if you want the annual option from the pricing page to actually work.

2.3 Get your API keys

1. Go to Developers > API keys.
2. Copy the Secret key (starts with sk_test_ while in test mode).

2.4 Set up the webhook

Stripe needs a way to tell your backend when someone actually completes checkout or cancels. This only works once your app is deployed with a real public URL (see the deployment section) — you can skip this step until then.

1. Go to Developers > Webhooks > Add endpoint.
2. Endpoint URL: https://<your-deployed-app-domain>/api/stripe/webhook
3. Select events to send: at minimum checkout.session.completed and customer.subscription.deleted.
4. After creating it, click into the endpoint and copy the Signing secret (starts with whsec_).

2.5 Test it before going live

Stripe's test mode gives you fake card numbers (4242 4242 4242 4242, any future expiry, any CVC) so you can run a full checkout without charging anyone. Only flip to Live Mode — and provide real bank details — once a full test purchase works end to end.

3. AI Provider Setup

ROOK's matching engine uses AI for résumé parsing, match explanations, and embeddings (architecture spec, sections 8 and 11). Pick one provider — Anthropic is what built this project, so it's a natural default, but OpenAI works equally well for this.

Anthropic

1. Go to console.anthropic.com and create an account.
2. Go to Settings > API Keys > Create Key.
3. Copy the key (starts with sk-ant-) into ANTHROPIC_API_KEY in your .env.
4. Add billing (Settings > Billing) — usage is pay-as-you-go; at the volumes described in the cost table below (Section 24 of the architecture spec), expect single-digit dollars per month early on.

4. Domain (Optional but Recommended)

You mentioned rookcareers.com as the target domain. Buying it now (roughly $10–$25/year through any registrar — Namecheap, Google Domains successor Squarespace Domains, or directly through DigitalOcean) means you can point it at your DigitalOcean app once deployed, and use a real domain for the Stripe webhook and success/cancel URLs instead of a placeholder.

5. Connecting Job Sources (Greenhouse / Lever / Ashby)

No signup, no API key — you're just identifying which companies use which ATS and what their board identifier is.

Finding a company's identifier

- Greenhouse: visit the company's careers page. If the URL looks like job-boards.greenhouse.io/COMPANYNAME or boards.greenhouse.io/COMPANYNAME, that COMPANYNAME is the board token.
- Lever: URL looks like jobs.lever.co/COMPANYNAME — that's the identifier.
- Ashby: URL looks like jobs.ashbyhq.com/COMPANYNAME — that's the job board name.

Not every company uses one of these three — many medical and veterinary employers use Workday or a custom career site instead, which the current adapters don't support yet (see the architecture spec, section 2, for the plan to add more adapter types later).

Adding an employer

For now, add rows directly in Supabase: Table Editor > employers > Insert row. Fill in company_name, company_slug (a URL-safe short name), ats_type (greenhouse, lever, or ashby), and ats_identifier (the value you found above). Leave active checked.

```
Example row: company_name: IDEXX Laboratories company_slug: idexx ats_type: greenhouse
ats_identifier: idexx active: true priority: high
```

Running ingestion

Once you have at least one employer row and your .env is filled in, run:

```
npm run ingest
```

This pulls every active employer's current job postings, normalizes them, and stores them in the jobs table. Run it manually first to confirm it works against 2–3 real companies before scheduling it (see the next section).

6. Scheduling Ingestion to Run Automatically

npm run ingest is a one-time run. For ROOK to actually stay current, it needs to run on a schedule — every few hours for high-priority employers, per the architecture spec (section 3). This backend does not include a scheduler yet; the simplest options, roughly in order of how little new infrastructure they require:

- DigitalOcean App Platform's own "Job" component type (separate from your web service) can run a command on a cron schedule against the same codebase you're already deploying.
- A free-tier cron service (e.g. cron-job.org) that hits a protected HTTP endpoint you add later to trigger ingestion.
- Trigger.dev or Inngest if you want proper retries, logging, and observability once this is running for real.

Whichever you choose, the command being scheduled is the same: node backend/ingest.js (or npm run ingest), with the same environment variables as the main app.

7. Environment Variables — Full Checklist

Variable	Where it comes from
SUPABASE_URL	Supabase > Project Settings > API > Project URL
SUPABASE_ANON_KEY	Supabase > Project Settings > API > anon public key
SUPABASE_SERVICE_ROLE_KEY	Supabase > Project Settings > API > service_role key (secret!)
STRIPE_SECRET_KEY	Stripe > Developers > API keys
STRIPE_PRICE_ID_MONTHLY	Stripe > Product Catalog > your $29/mo price
STRIPE_WEBHOOK_SECRET	Stripe > Developers > Webhooks > your endpoint > Signing secret
ANTHROPIC_API_KEY	console.anthropic.com > Settings > API Keys
PUBLIC_APP_URL	Your deployed app's URL (or http://localhost:8080 for local testing)
PORT	8080 locally; DigitalOcean sets this automatically in production

All of these go in a file named .env in the project root (copy .env.example to .env and fill it in). Never commit the real .env file to GitHub — it's already listed in .gitignore.

8. Deploying to DigitalOcean App Platform

1. Push the project to a GitHub repo (or paste the files in via the GitHub web editor, same as your other projects).
2. In DigitalOcean, go to App Platform > Create App > connect the GitHub repo.
3. It should auto-detect Node.js from package.json. Build command: npm install. Run command: npm start.
4. Under the app's Settings, add every variable from the checklist above as an Environment Variable (mark the Supabase service role key and Stripe secret key as "Encrypted").
5. Set PUBLIC_APP_URL to your actual DigitalOcean app URL (or your custom domain once connected).
6. Deploy. Once it's live, go back to Stripe and finish the webhook setup from Section 2.4 using the real URL.
7. If you bought a domain, point it at the app under Settings > Domains, then update PUBLIC_APP_URL again to match.

9. Testing Checklist Before You Call It Launched

- Homepage and every page load correctly at the deployed URL
- Running npm run ingest locally successfully pulls jobs from at least 2–3 real employers into Supabase
- A test Stripe checkout (test-mode card 4242 4242 4242 4242) completes and the webhook updates the candidate's subscription_status
- A résumé file uploaded through the API actually appears in the Supabase resumes storage bucket
- Row-level security is working: one test user cannot read another test user's candidate_profiles row

What's Still Not Built

Being direct about where this leaves off: this guide gets the accounts, database, and job-ingestion pipeline working. It does NOT yet include the AI matching engine itself (scoring, explanations, embeddings), résumé text extraction/parsing, the tailored-résumé generator, or wiring the 13 existing frontend pages to call any of these real endpoints instead of showing sample data. Those are exactly the Phase 1 / Phase 1.5 items in ROOK-Technical-Architecture-Spec.docx — the logical next block of work once everything in this guide is confirmed working.